

7.2.5 答案与解析

一、单项选择题

01. A

顺序查找是指从表的一端开始向另一端查找。它不要求查找表具有随机存取的特性，可以是顺序存储结构或链式存储结构。

02. B

对于顺序查找，不管线性表是有序的还是无序的，成功查找第一个元素的比较次数为 1，成

功查找第二个元素的比较次数为 2, 以此类推, 即每个元素查找成功的比较次数只与其位置有关 (与是否有序无关), 因此查找成功的平均时间两者相同。

03. B

在有序单链表上做顺序查找, 查找成功的平均查找长度与在无序顺序表或有序顺序表上做顺序查找的平均查找长度相同, 都是 $(n+1)/2$ 。

04. A

在长度为 3 的顺序表中, 查找第一个元素的查找长度为 1, 查找第二个元素的查找长度为 2, 查找第三个元素的查找长度为 3, 所以有

$$ASL_{成功} = \frac{1}{2} \times 1 + \frac{1}{3} \times 2 + \frac{1}{6} \times 3 = \frac{5}{3}$$

05. D

二分查找通过下标来定位中间位置元素, 所以应采用顺序存储, 且二分查找能够进行的前提是查找表是有序的, 但具体是从大到小还是从小到大的顺序则不做要求。

06. C

折半查找的快体现在一般情况下, 在大部分情况下要快, 但是对于某些特殊情况, 顺序查找可能会快于折半查找。例如, 查找一个含 1000 个元素的有序表中的第一个元素时, 顺序查找的比较次数为 1 次, 而折半查找的比较次数却将近 10 次。

07. B

A 显然排除。对于选项 C, 考点精析示例中的判定树就不是完全二叉树。由选项 C 也可排除选项 D, 且满二叉树对结点数有要求。只可能选 B。事实上, 由折半查找的定义不难看出, 每次把一个数组从中间结点分割时, 总是把数组分为结点数相差最多不超过 1 的两个子数组, 从而使得对应的判定树的两棵子树高度差的绝对值不超过 1, 所以应是平衡二叉树。

08. B

折半查找的性能分析可以用二叉判定树来衡量, 平均查找长度和最大查找长度都是 $O(\log_2 n)$; 二叉排序树的查找性能与数据的输入顺序有关, 最好情况下的平均查找长度与折半查找相同, 但最坏情况即形成单支树时, 其查找长度为 $O(n)$ 。

09. B

依据折半查找算法的思想, 第一次 $mid = \lfloor (1+11)/2 \rfloor = 6$, 第二次 $mid = \lfloor [(6+1)+11]/2 \rfloor = 9$, 第三次 $mid = \lfloor [(9+1)+11]/2 \rfloor = 10$, 第四次 $mid = 11$ 。

10. B

开始时 low 指向 13, high 指向 134, mid 指向 50, 比较第一次 $90 > 50$, 所以将 low 指向 62, high 指向 134, mid 指向 90, 第二次比较找到 90。

11. A

在折半查找算法中, mid 取值的方式是确定的, 要么采用向上取整, 要么采用向下取整, 而不能出现两种情况。对于 A, 第 1 次比较的元素是 f , 为向下取整; 第 2 次比较的元素是 c , 为向下取整; 第 3 次比较的元素是 b , 为向下取整, 查找成功, 符合二分查找。对于 B, 第 1 次比较的元素是 f , 为向下取整; 第 2 次比较的元素是 d , 为向上取整, 两次 mid 取值的方式不同, 不符合二分查找。对于 C, 第 1 次比较的元素是 g , 为向上取整; 第 2 次比较的元素是 c , 为向下取整, 不符合二分查找。对于 D, 第 1 次比较的元素是 g , 为向上取整; 第 2 次比较的元素是 d , 为正中间元素; 第 3 次比较的元素为 b , 为向下取整, 不符合二分查找。

12. A

对 n 个结点的判定树, 设结点总数 $n = 2^h - 1$, 则 $h = \lceil \log_2(n+1) \rceil$ 。

另解：特殊值代入法。直接将 $n=1$ 和 $n=2$ 的情况代入，仅有 A 满足要求。

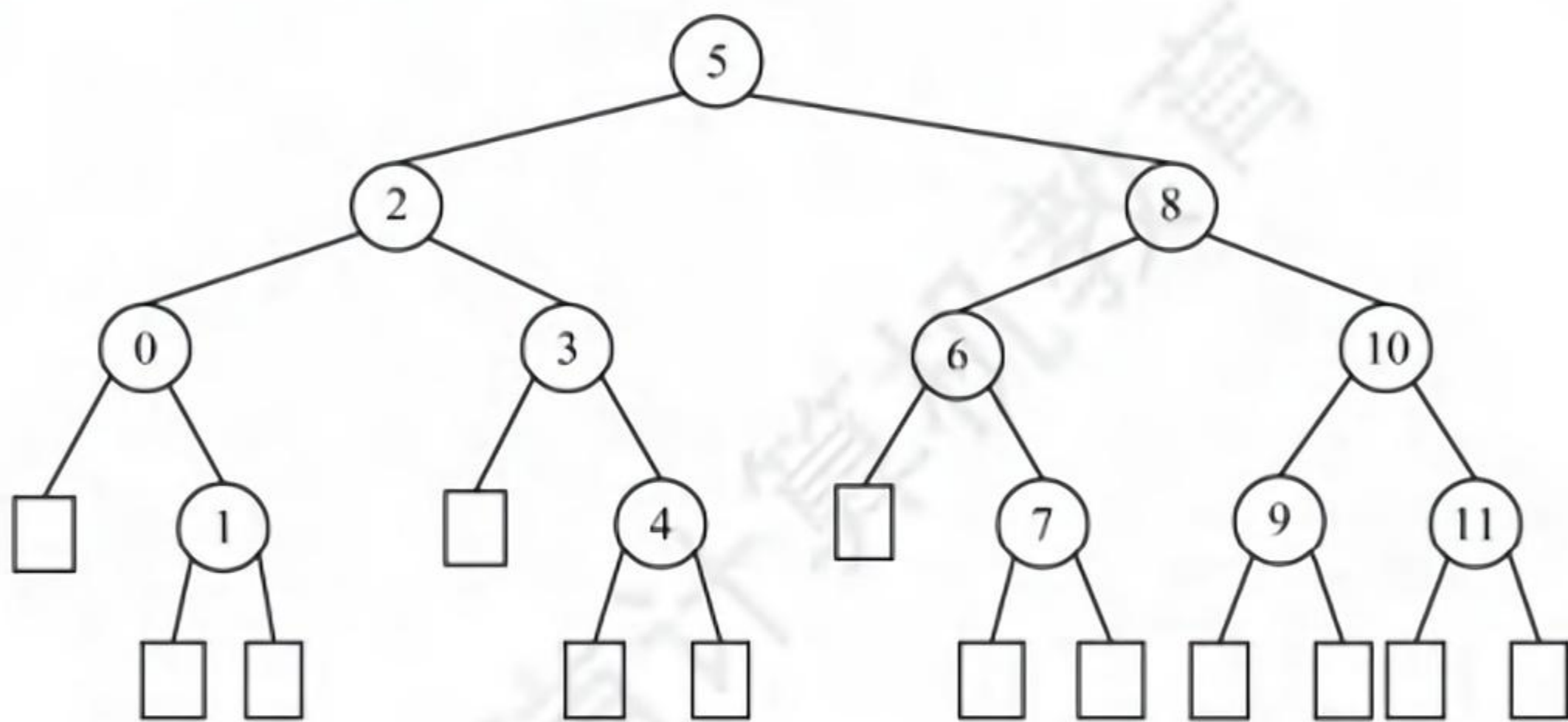
13. A、B

对于此类题，有两种做法：一种方法是，画出查找过程中构成的判定树，让最小的分支高度对应于最少的比较次数，让最大的分支高度对应于最多的比较次数，出现类似于长度为 15 的顺序表时，判定树刚好是一棵满树，此时最多比较次数与最少比较次数相等；另一种方法是，直接用公式求出最小的分支高度和最大分支高度，从前面的讲解不难看出最大分支高度为 $H = \lceil \log_2(n+1) \rceil = 5$ ，这对应的就是最多比较次数，然后由于判定树不是一棵满树，所以至少应该是 4（由判定树的各分支高度最多相差 1 得出）。

注意，若是求查找成功或查找失败的平均查找长度，则需要画出判定树进行求解。此外，对长度为 n 的有序表，采用折半查找时，查找成功和查找失败的最多比较次数相同，均为 $\lceil \log_2(n+1) \rceil$ 。

14. A、D

假设有序表中元素为 $A[0 \dots 11]$ ，不难画出对它进行折半查找的判定树如下图所示，圆圈是查找成功结点，方形是虚构的查找失败结点。从而可以求出查找成功的 $ASL = (1 + 2 \times 2 + 3 \times 4 + 4 \times 5) / 12 = 37/12$ ，查找失败的 $ASL = (3 \times 3 + 4 \times 10) / 13$ 。



注意

对于本类题目，应先根据所给 n 的值，画出如上图所示的折半查找判定树。另外，查找失败结点的 ASL 不是到图中的方形结点，而是到方形结点上一层的圆形结点。

15. D

折半查找法不仅要求数据元素有序，而且要求必须为顺序存储，A 错误。折半查找法在最坏情况下的时间性能为 $O(\log_2 n)$ ，二叉查找树在最坏情况下的时间性能为 $O(n)$ ，B 错误。在每个元素查找概率不同的情况下，折半查找法的平均查找长度可能大于顺序查找法，C 错误。

16. B

通常情况下，在分块查找的结构中，不要求每个索引块中的元素个数都相等。

17. A

设块长为 b ，索引表包含 n/b 项，索引表的 $ASL = (n/b + 1)/2$ ，块内的 $ASL = (b + 1)/2$ ，总 $ASL =$ 索引表的 $ASL +$ 块内的 $ASL = (b + n/b + 2)/2$ ，其中对于 $b + n/b$ ，由均值不等式知 $b = n/b$ 时有最小值，此时 $b = \sqrt{n}$ 。则最理想块长为 $\sqrt{2500} = 50$ 。

18. B

根据公式 $ASL = L_1 + L_s = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2 + 2s + n}{2s}$ ，其中 $b = n/s$, $s = 123/3$, $n = 123$ ，代入不难得出 ASL 为 23。所以选 B。另一方面，可根据穷举法来一步步模拟。对于 A 块中的元素，查找过程

的第一步是先找到 A 块, 由于是顺序查找, 找到 A 块只需一步, 然后在 A 块中顺序查找。因此, A 块内各元素查找长度分别为 2, 3, 4, ..., 42。对于 B 块, 采用类似的方法, 但查找到 B 块要比查找到 A 块多一步, 因此 B 块内各元素查找长度为 3, 4, 5, ..., 43。同理, C 块中各个元素查找长度为 4, 5, 6, ..., 44。所以平均查找长度为 $(2 + 3 + 4 + \dots + 42 + 3 + 4 + 5 + \dots + 43 + 4 + 5 + 6 + \dots + 44) / 123 = 23$ 。

19. C

为使查找效率最高, 每个索引块的大小应是 $\sqrt{65025} = 255$, 为每个块建立索引, 则索引表中索引项的个数为 255。若对索引项和索引块内部都采用折半查找, 则查找效率最高, 为 $\lceil \log_2(255+1) \rceil + \lceil \log_2(255+1) \rceil = 16$ 。

20. B

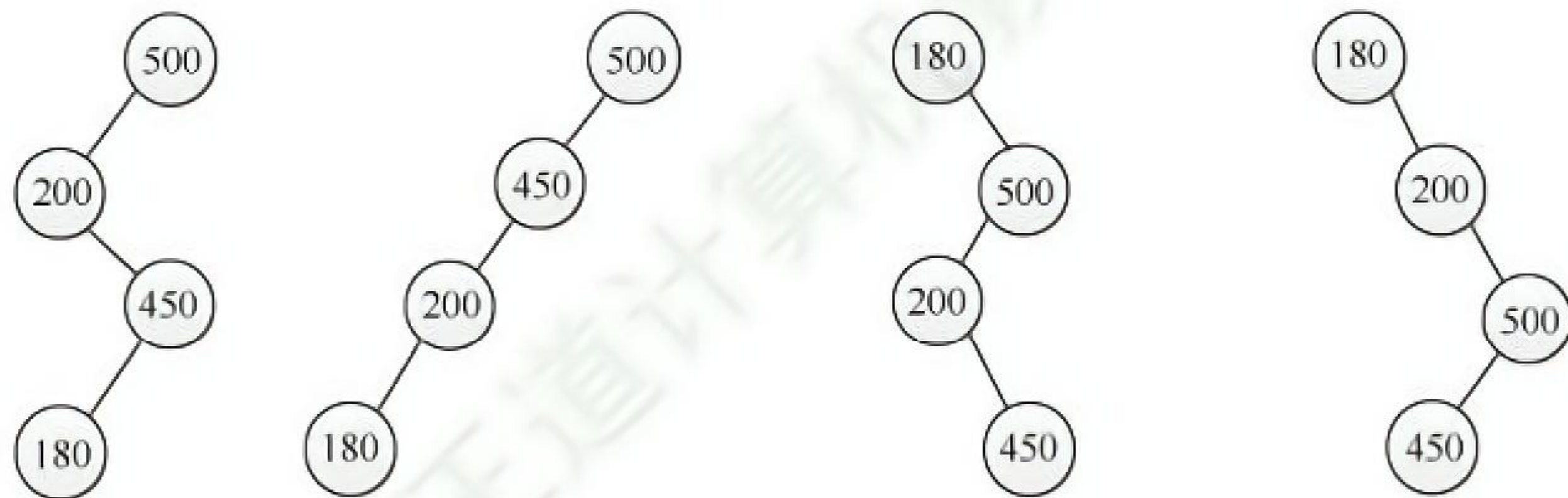
折半查找法在查找不成功时和给定值进行关键字的比较次数最多为树的高度, 即 $\lfloor \log_2 n \rfloor + 1$ 或 $\lceil \log_2(n+1) \rceil$ 。在本题中, $n = 16$, 所以比较次数最多为 5。

注意

在折半查找判定树中的方形结点是虚构的, 它不计入比较的次数。

21. A

画出查找路径图, 因为折半查找判定树是一棵二叉排序树, 看其是否满足二叉排序树的要求。



显然, 选项 A 的查找路径不满足。

22. B

本题为送分题。

该程序采用跳跃式的顺序查找法查找升序数组中的 x 。显然, x 越靠前, 比较次数越少。

23. A

对于给定的一个有序查找表, 其对应的折半查找判定树是确定且唯一的。7.2.2 节描述的折半查找算法中, $mid = \lfloor (low+high)/2 \rfloor$, 因此若表中初始有 $2n+1$ 个元素, 则 mid 分割后, 左右子树各有 n 个元素; 若表中初始有 $2n$ 个元素, 则 mid 分割后, 左子树有 $n-1$ 个元素, 右子树有 n 个元素。即左子树的元素个数或者与右子树的元素个数相等, 或者比右子树少一个。若令 $mid = \lceil (low+high)/2 \rceil$, 不难理解, 左子树的元素个数或者与右子树的元素个数相等, 或者比右子树多一个。选项 A, 树中每个左子树都与右子树的结点个数相等, 或者多一个结点, 符合向上取整的规则。选项 B、C、D, 存在有的左子树比右子树多一个结点, 有的左子树比右子树少一个结点, 不符合折半查找的规则。

24. B

用折半查找法查找给定值的比较次数最多不超过折半查找判定树的高度。折半查找判定树的树高 $h = \lceil \log_2(n+1) \rceil$, 将 $n = 600$ 代入, 结果为 10。

7.3.5 答案与解析

一、单项选择题

01. C

二叉排序树插入新结点时不会引起树的分裂组合。对二叉排序树进行中序遍历可得到有序序列。当插入的关键字有序时，二叉排序树会形成一个长链，此时深度最大。在此种情况下进行查找，有可能需要比较每个结点的关键字，超过总结点数的 $1/2$ 。

02. B

由二叉排序树的定义不难得出中序遍历二叉树得到的序列是一个有序序列。

03. A

二叉排序树的查找路径是自顶向下的，其平均查找长度主要取决于树的高度。

04. B

在二叉排序树的存储结构中，每个结点由三部分构成，其中左（或右）指针指向比该结点的关键字值小（或大）的结点。关键字值最大的结点位于二叉排序树的最右位置，因此它的右指针一定为空（有可能不是叶结点）。还可用反证法，若右指针不为空，则右指针上的关键字肯定比原关键字大，所以原关键字结点一定不是值最大的，与条件矛盾，所以右指针一定为空。

05. C

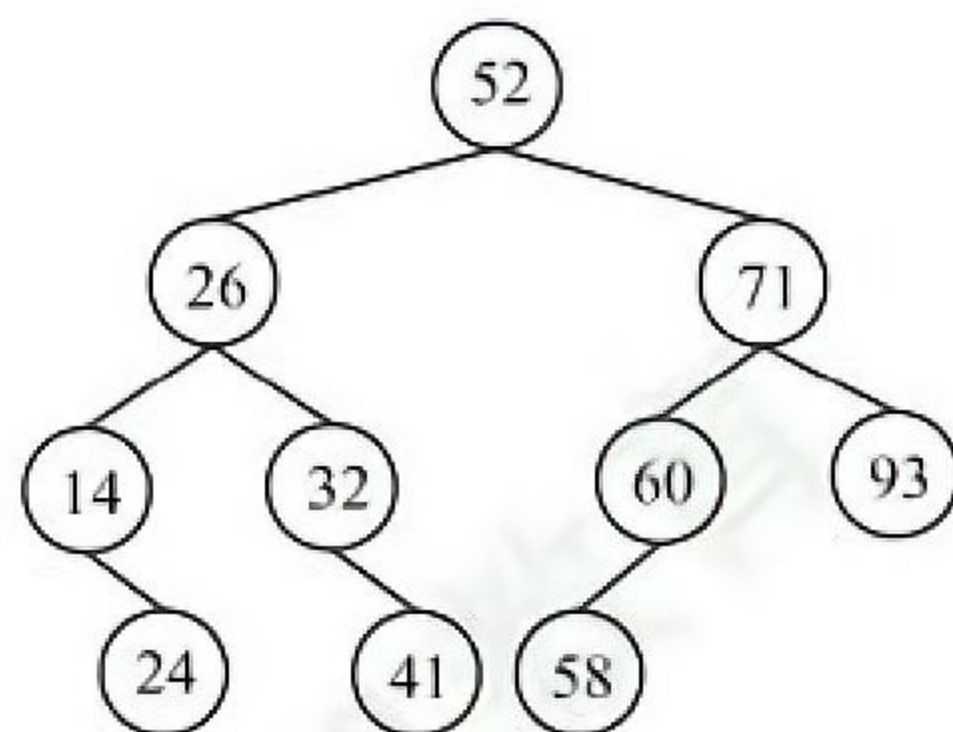
在二叉排序树上查找时，先与根结点值进行比较，若相同，则查找结束，否则根据比较结果，沿着左子树或右子树向下继续查找。根据二叉排序树的定义，有左子树结点值 $\leq$ 根结点值 $\leq$ 右子树结点值。C 序列中，比较 911 关键字后，应转向其左子树比较 240，左子树中不应出现比 911 更大的数值，但 240 竟有一个右孩子结点值为 912，所以不可能是正确的序列。

06. C

按照二叉排序树的构造方法，不难得到 A、B、D 序列的构造结果相同。

07. A

以第一个元素为根结点，依次将元素插入树，生成的二叉排序树如下图所示。进行查找时，先与根结点比较，然后根据比较结果，继续在左子树或右子树上进行查找。比较的结点依次为 52, 71, 60。



08. D

当输入序列是一个有序序列时，构造的二叉排序树是一个单支树，当查找一个不存在的关键

字值或最后一个结点的关键字值时, 需要 n 次比较。

09. D

五个不同结点构造的二叉查找树, 中序序列是确定的。先序序列的个数为 $n=5$ 的卡特兰数, 加上中序序列和先序序列能唯一确定一棵二叉树, 因此二叉排序树的形态共有 $\text{Catalan}(5)=42$ 种。

10. D

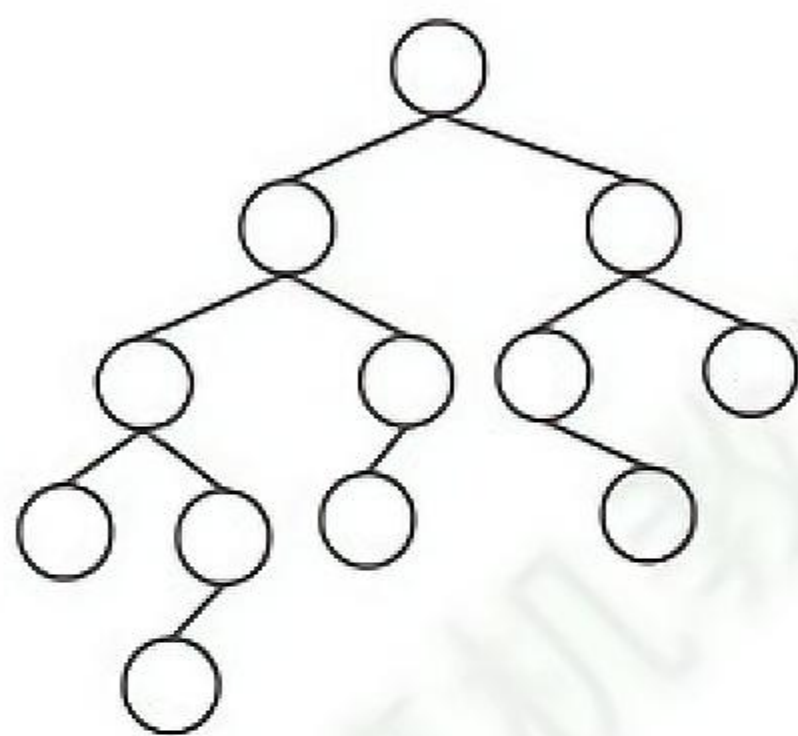
当二叉排序树的叶结点全部都在相邻的两层内时, 深度最小。理想情况是从第一层到倒数第二层为满二叉树。类比完全二叉树, 可得深度为 $\lceil \log_2(n+1) \rceil$ 。

11. C

平衡二叉树结点数的递推公式为 $n_0=0$, $n_1=1$, $n_2=2$, $n_h=1+n_{h-1}+n_{h-2}$ (h 为平衡二叉树高度, n_h 为构造此高度的平衡二叉树所需的最少结点数)。通过递推公式可得, 构造 5 层平衡二叉树至少需 12 个结点, 构造 6 层至少需要 20 个结点。

12. B

设 n_h 表示高度为 h 的平衡二叉树中含有的最少结点数, 则有 $n_1=1$, $n_2=2$, $n_h=n_{h-1}+n_{h-2}+1$, 由此求出 $n_5=12$, 对应的 AVL 如下图所示。



13. D

高度为 3 的平衡二叉树的左右子树的高度共有三种情况: ①左右子树都是高度为 2 的平衡二叉树; ②左子树是高度为 1 的平衡二叉树, 右子树是高度为 2 的平衡二叉树; ③左子树是高度为 2 的平衡二叉树, 右子树是高度为 1 的平衡二叉树。高度为 1 的平衡二叉树只有 1 种形态, 即单个结点, 如图 1 所示; 高度为 2 的平衡二叉树有 3 种形态, 如图 2 所示。



图 1

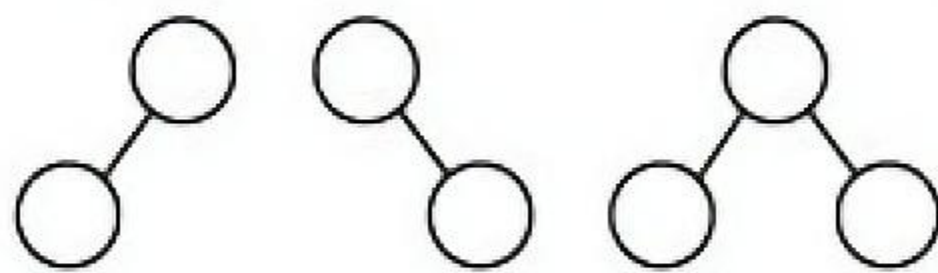


图 2

因此, 对于情况①, 共有 $3 \times 3 = 9$ 种树形态; 对于情况②, 共有 $1 \times 3 = 3$ 种树形态; 情况③和情况②类似, 也有 3 种树形态, 所以共有 $9 + 3 + 3 = 15$ 种树形态。

14. A

插入和删除结点都有可能引起 LR 平衡旋转或者 RL 平衡旋转, 发生两次旋转操作。

15. C

当按关键字有序的顺序插入初始为空的平衡二叉树时, 若关键字个数 $n=2^k-1$ 时, 则该平衡二叉树一定是一棵满二叉树 (可以用 1~3、1~7 手工验证)。当插入关键字 1023 时, 平衡二叉树正好是一棵满二叉树, 高度是 9。因此, 插入关键字 1024 后, 平衡二叉树的高度是 10。

16. D

I 和 II 都是红黑树的性质。AVL 是高度平衡的二叉查找树, 红黑树是适度平衡的二叉查找树, 从这一点也可以看出 AVL 的查询效率往往更优, III 错误。AVL 的某结点的左右孩子的平衡因子都是零, 并不能说明左右子树的高度相等, 因此该结点的平衡因子不一定为零, IV 错误。

17. C

自平衡的二叉排序树是指在插入和删除时能自动调整以保持其所定义的平衡性，两者都属于自平衡二叉树，A 正确。两者的查找、插入、删除操作的时间复杂度都为 $O(\log_2 n)$ ，B 正确。在红黑树中删除结点时，情况 1 可能变为情况 2、3 或 4，情况 2 会变为情况 3，可能会出现旋转次数超过 2 次的情况，C 错误。从任一结点到每个叶结点的所有路径都包含相同数目的黑结点，没有两个连续的红结点，且叶结点是红色的，这意味着在任一结点到其左右子树中最远和最近的叶结点之间，红结点的数目小于或等于黑结点的数目，路径长度之比不超过 2，D 正确。

18. B

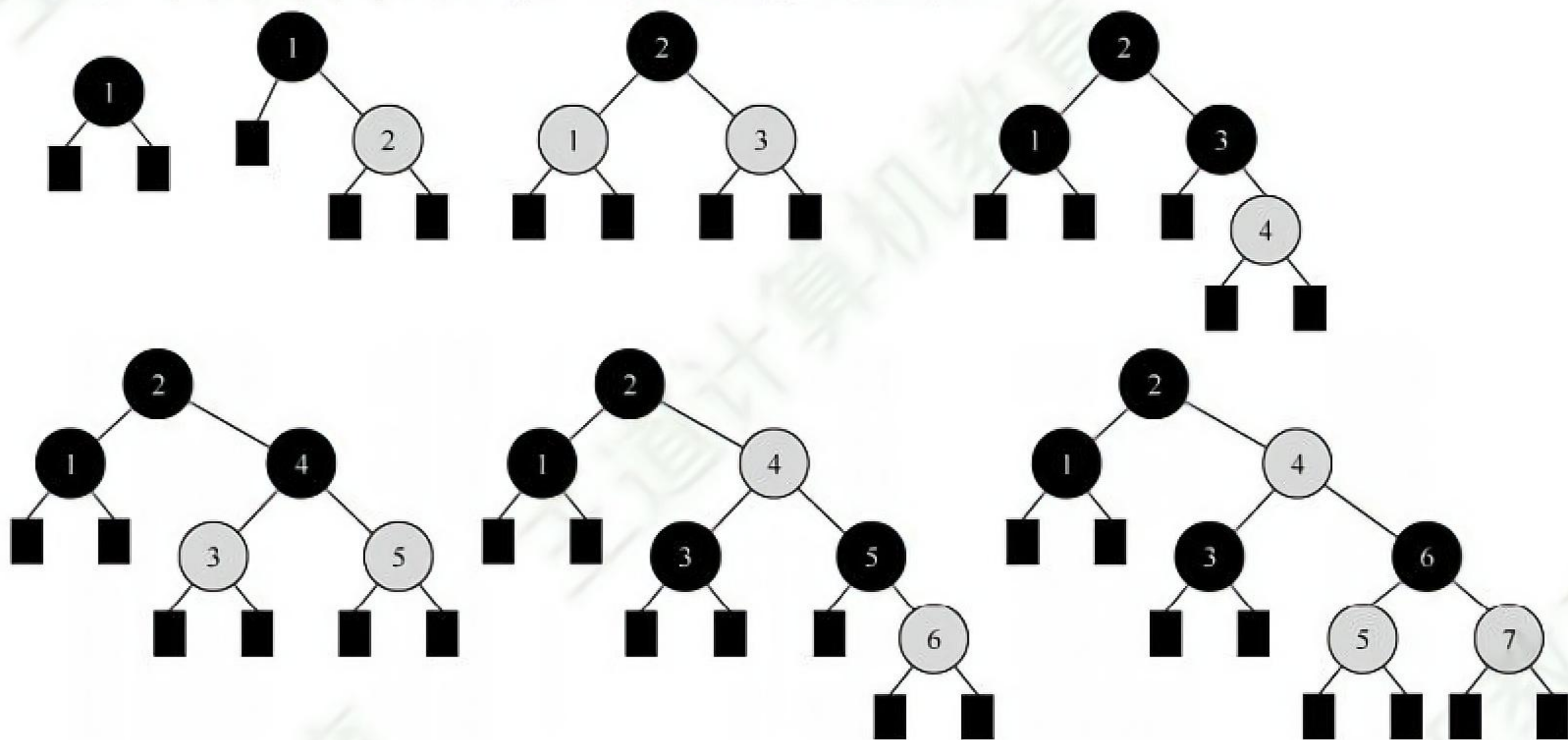
红黑树的红结点数目最大可以是黑结点数目的 2 倍（如一棵有 3 个结点的红黑树，第 1 层为黑色，第 2 层为红色），A 错误。从根结点出发到所有叶结点的黑结点数是相同的，若所有结点都是黑色，则一定是满二叉树，B 正确。考虑某个黑结点，它可以有一个空叶结点孩子和一个非空红结点孩子，C 错误。红黑树中可能存在红结点，根结点为红色的子树不是红黑树，D 错误。

19. A

红黑树是一种特殊的二叉排序树，B 项不满足二叉排序树的性质。C 项中，结点 2 的左右黑结点数不同。D 项中，结点 3 的左右黑结点数不同。只有 A 项满足红黑树的定义。

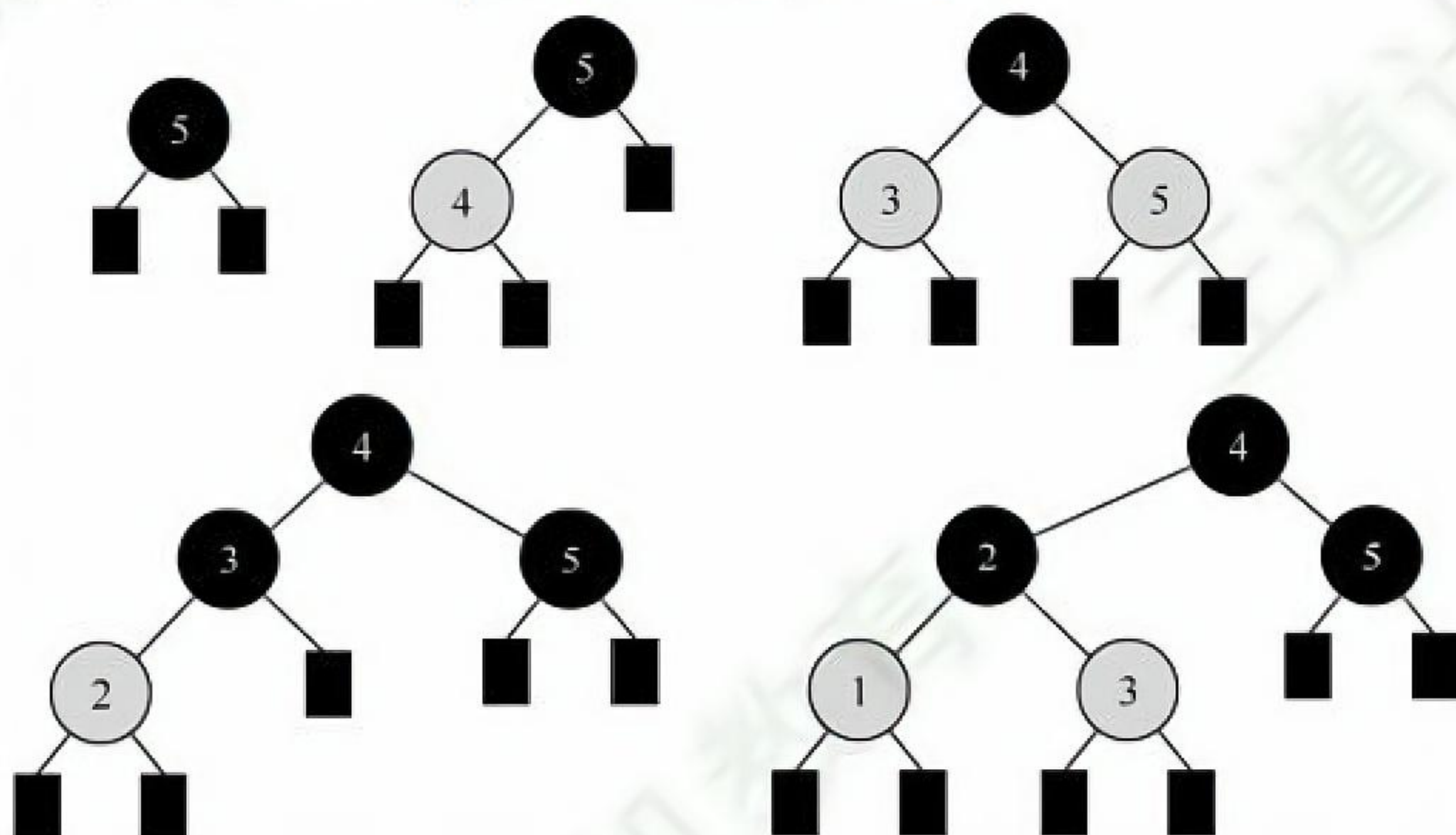
20. C

关键字 1, 2, 3, 4, 5, 6, 7 依次插入红黑树后的形态变化如下：



21. D

关键字 5, 4, 3, 2, 1 依次插入红黑树后的形态变化如下：



22. B

插入结点 2 且将其染成红色后违反不红红原则，并且叔结点是黑色，应进行 LL 旋转，将结

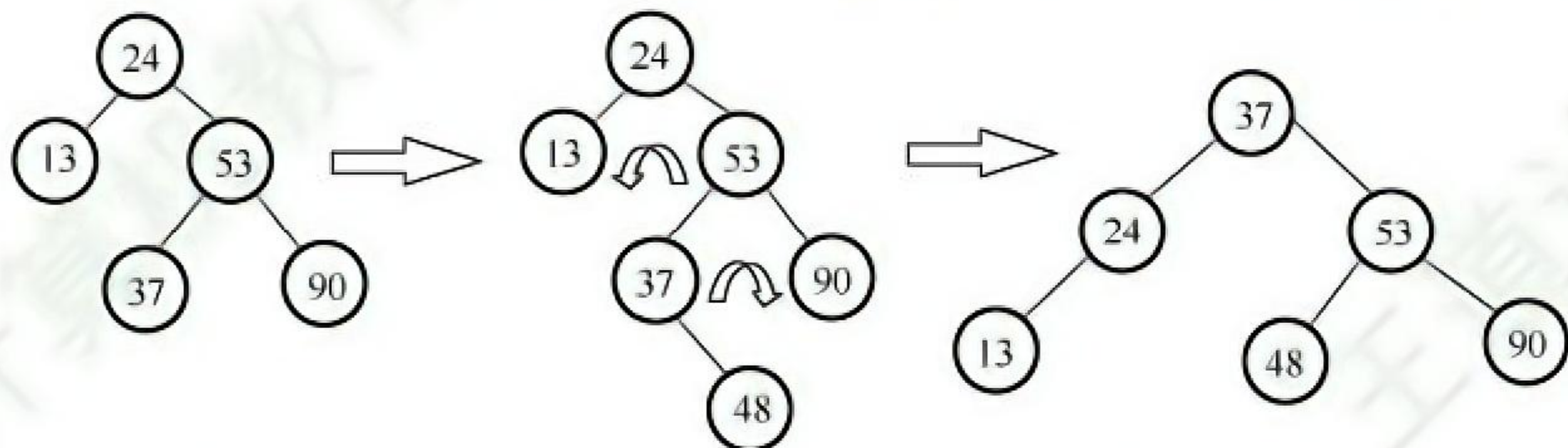
点 3 右旋旋转到结点 5 的位置, 结点 2 和结点 5 分别成为结点 3 的左、右孩子, 然后将结点 3 染成黑色, 结点 2 和结点 5 染成红色。因此, 下一步应进行右旋操作。

23. B

根据平衡二叉树的定义, 任意结点的左、右子树高度差的绝对值不超过 1。而其余 3 个答案均可以找到不满足条件的结点。答题时可以把每个非叶结点的平衡因子都写出来。

24. C

插入 48 以后, 该二叉树根结点的平衡因子由 -1 变为 -2, 在最小不平衡子树根结点的右子树 (R) 的左子树 (L) 中插入新结点引起的不平衡属于 RL 型平衡旋转 (先右旋后左旋)。

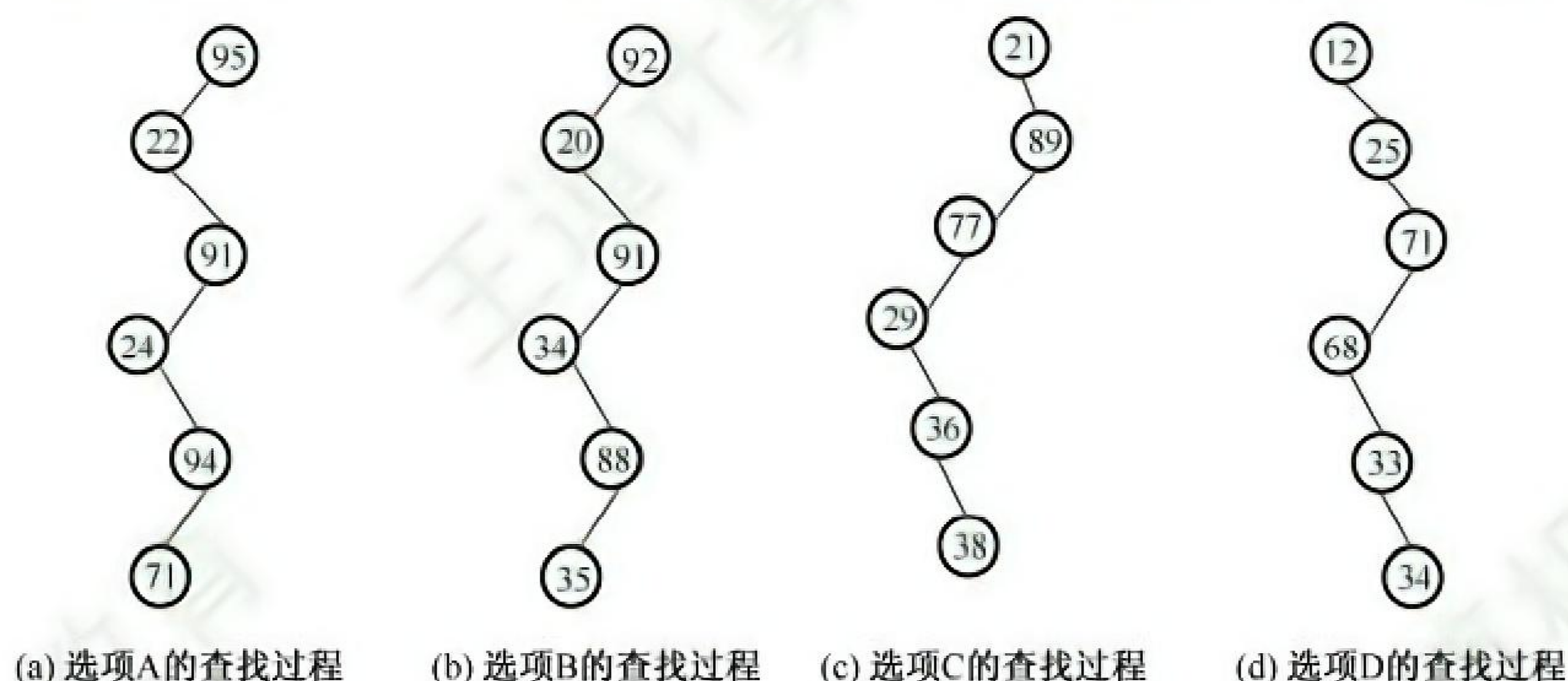


调整后, 关键字 37 所在结点的左、右子结点中保存的关键字分别是 24、53。

25. A

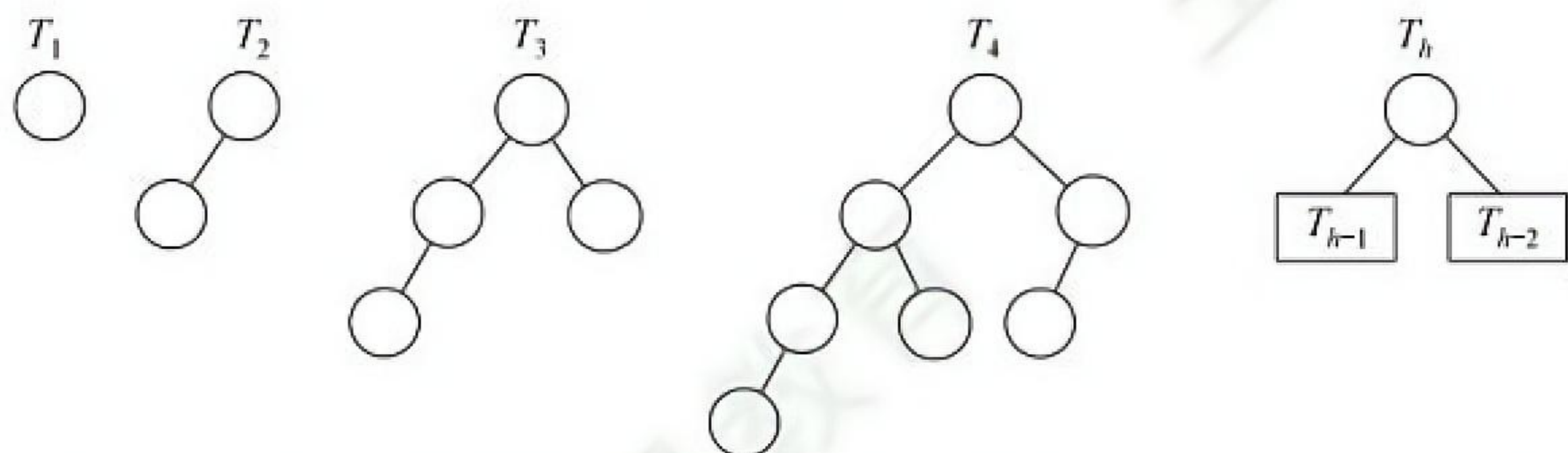
在二叉排序树中, 左子树结点值小于根结点, 右子树结点值大于根结点。在选项 A 中, 当查找到 91 后再向 24 查找, 说明这一条路径 (左子树) 之后查找的数都要比 91 小, 而后面却查找到了 94 (解题过程中, 建议配合画图), 因此错误。

画图法: 各选项对应的查找过程如下图, 选项 B、C、D 对应的查找树都是二叉排序树, 选项 A 对应的查找树不是二叉排序树, 因为在 91 为根的左子树中出现了比 91 大点的结点 94。



26. B

所有非叶结点的平衡因子均为 1, 即平衡二叉树满足平衡的最少结点情况, 如下图所示。对于高度为 n 、左右子树的高度分别为 $n-1$ 和 $n-2$ 、所有非叶结点的平衡因子均为 1 的平衡二叉树, 计算总结点数的公式为 $C_n = C_{n-1} + C_{n-2} + 1$, $C_1 = 1$, $C_2 = 2$, $C_3 = 2 + 1 + 1 = 4$, 可推出 $C_6 = 20$ 。



画图法: 先画出 T_1 和 T_2 ; 然后新建一个根结点, 连接 T_2 、 T_1 构成 T_3 ; 新建一个根结点, 连接 T_3 、 T_2 构成 T_4 ……直到画出 T_6 , 可知 T_6 的结点数为 20。

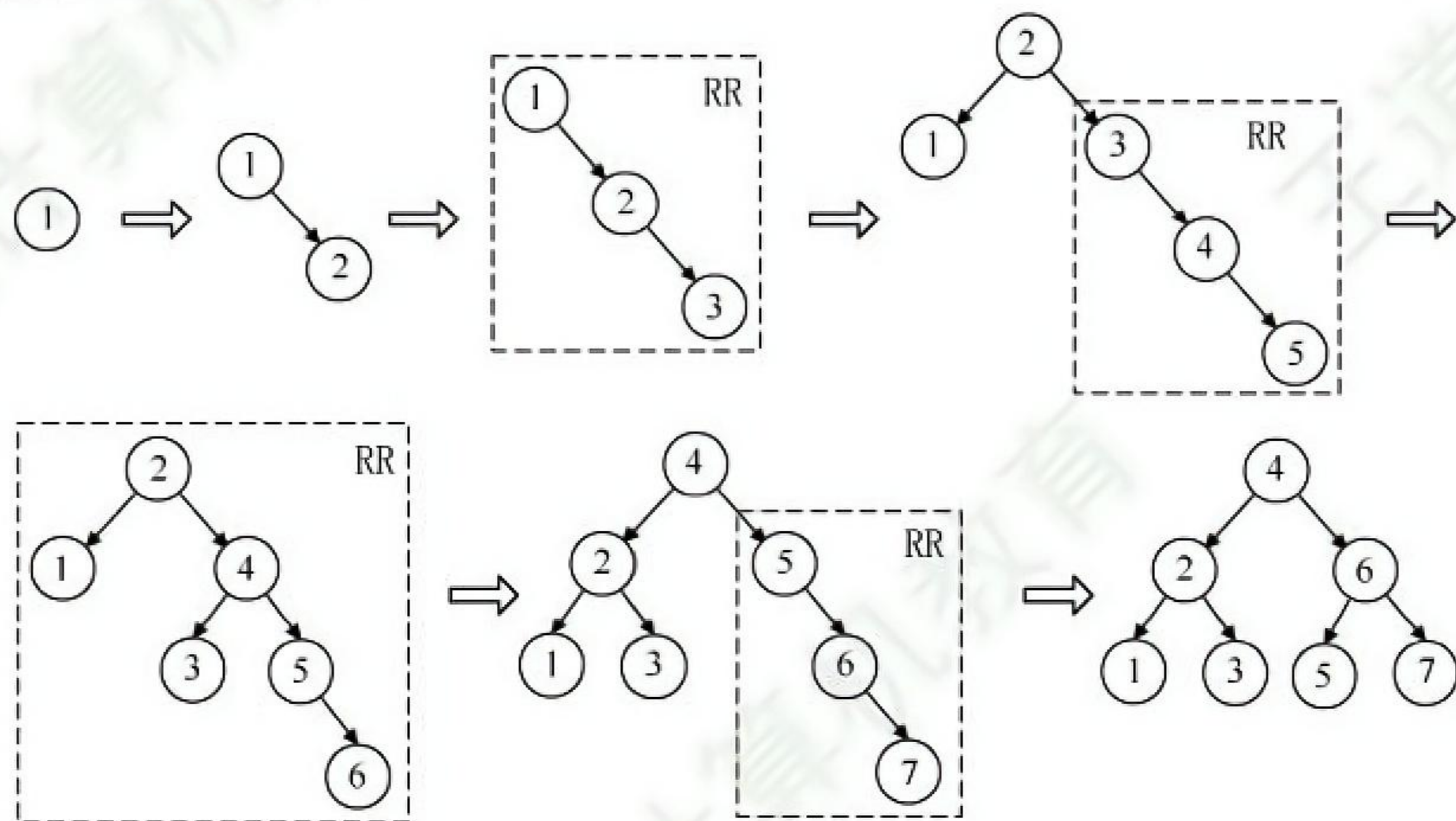
排除法：对于 A，高度为 6、结点数为 12 的树怎么也无法达到平衡。对于 C，结点较多时，考虑较极端的情形，即第 6 层只有最左叶子的完全二叉树刚好有 32 个结点，虽然满足平衡的条件，但显然再删去部分结点依然不影响平衡，不是最少结点的情况。同理 D 错误。

27. C

由于在二叉排序树中插入结点的位置是一个新的叶结点，若删除的是叶结点，则重新插入后得到的二叉排序树与原来的二叉排序树相同。若删除的是非叶结点，在删除过程中会找其他结点填补，重新插入后变成叶结点，则得到的二叉排序树与原来的二叉排序树不同。

28. D

利用 7 个关键字构建平衡二叉树 T ，平衡因子为 0 的分支结点个数为 3，构建的平衡二叉树及构造与调整过程如下图所示。



29. D

大多数教材将平衡二叉树定义为一种高度平衡的二叉排序树，二叉排序树的中序序列是一个升序序列，而题意正好相反。由此可知，命题老师认为平衡二叉树仅为一棵满足高度平衡的二叉树，不一定是二叉排序树。只有两个结点的平衡二叉树的根结点的度为 1，A 错误。中序遍历后得到一个降序序列（与二叉排序树正好相反），树中最大元素一定无左子树（可能有右子树），这与二叉排序树也正好相反，也因此不一定是叶结点，B 错误，D 正确。最后插入的结点可能会导致平衡调整，而不一定是叶结点，C 错误。

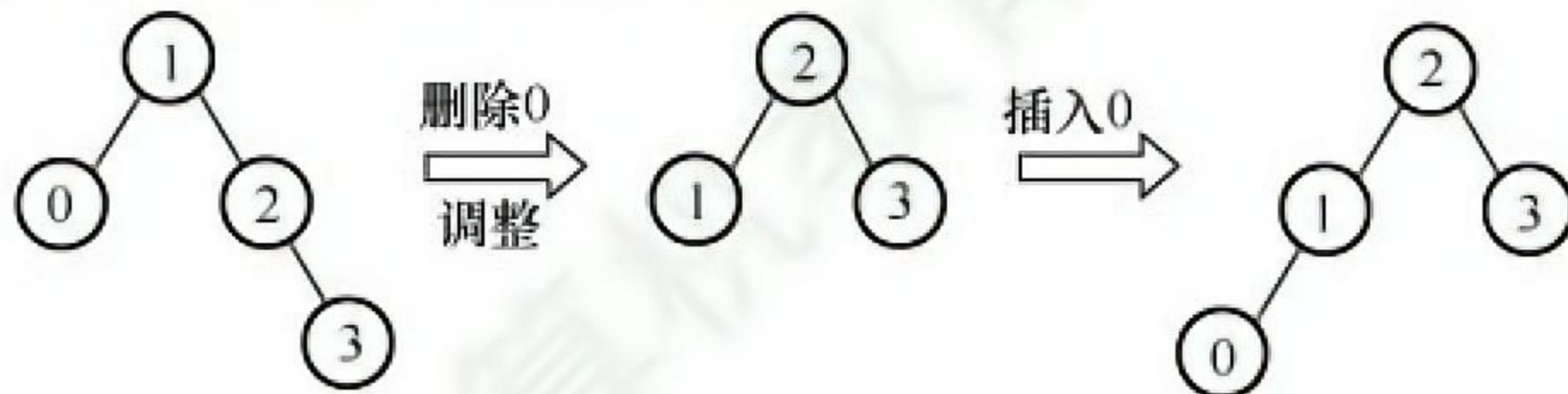
30. C

根据二叉排序树的特性：中序遍历（LNR）得到的是一个递增序列。图中二叉排序树的中序遍历序列为 x_1, x_3, x_5, x_4, x_2 ，可知 $x_3 < x_5 < x_4$ 。

31. A

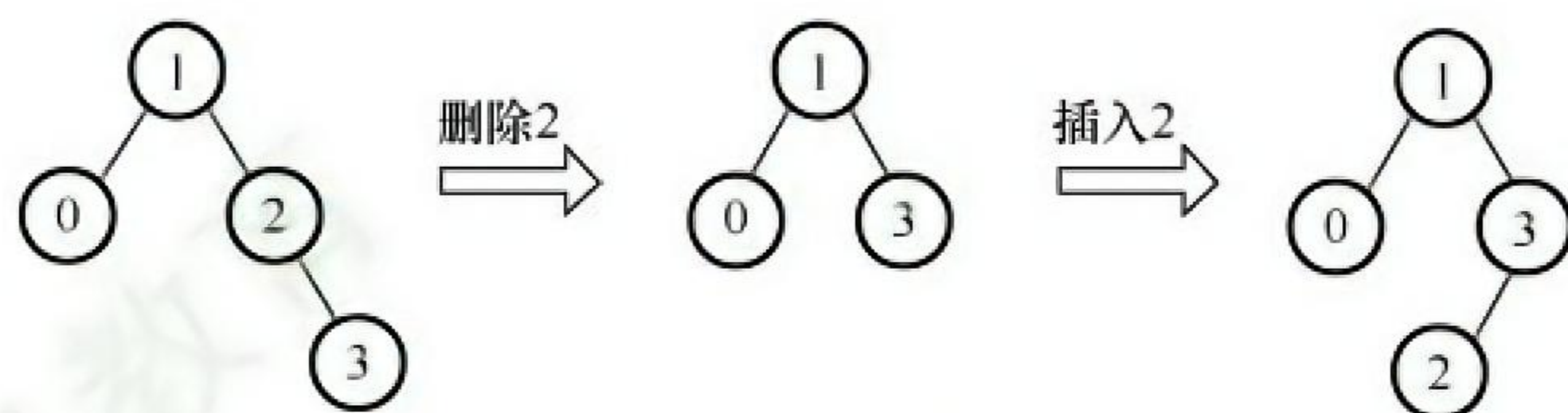
在非空平衡二叉树中插入结点，在失去平衡调整前，一定插入在叶结点的位置。

若删除的是 T_1 的叶结点，则删除后平衡二叉树可能不会失去平衡，即不会发生调整，再插入此结点得到的二叉平衡树 T_1 与 T_3 相同；若删除后平衡二叉树失去平衡而发生调整，再插入结点得到的二叉平衡树 T_3 与 T_1 可能不同。说法 I 正确。例如，如下图所示，删除结点 0，平衡二叉树失衡调整，再插入结点 0 后，平衡二叉树和以前不同。

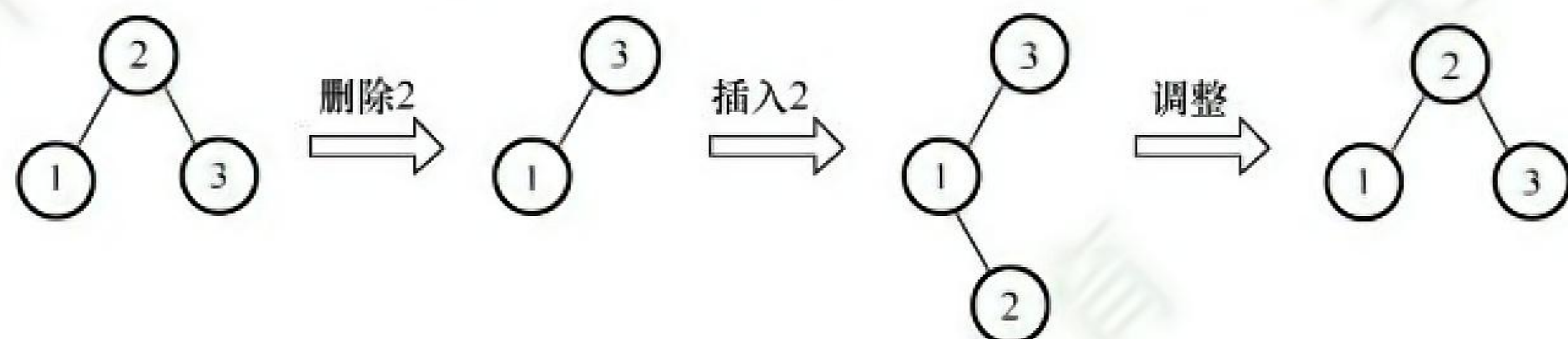


对于比较绝对的说法 II 和 III，通常只需举出反例即可。

若删除的是 T_1 的非叶结点，且删除和插入操作均没有导致平衡二叉树的调整（这时可以首先想到删除的结点只有一个孩子的情况），则该结点从非叶结点变成了叶结点， T_1 与 T_3 显然不同。例如，如下图所示，删除结点 2，用右孩子结点 3 填补，再插入结点 2，平衡二叉树和以前不同。

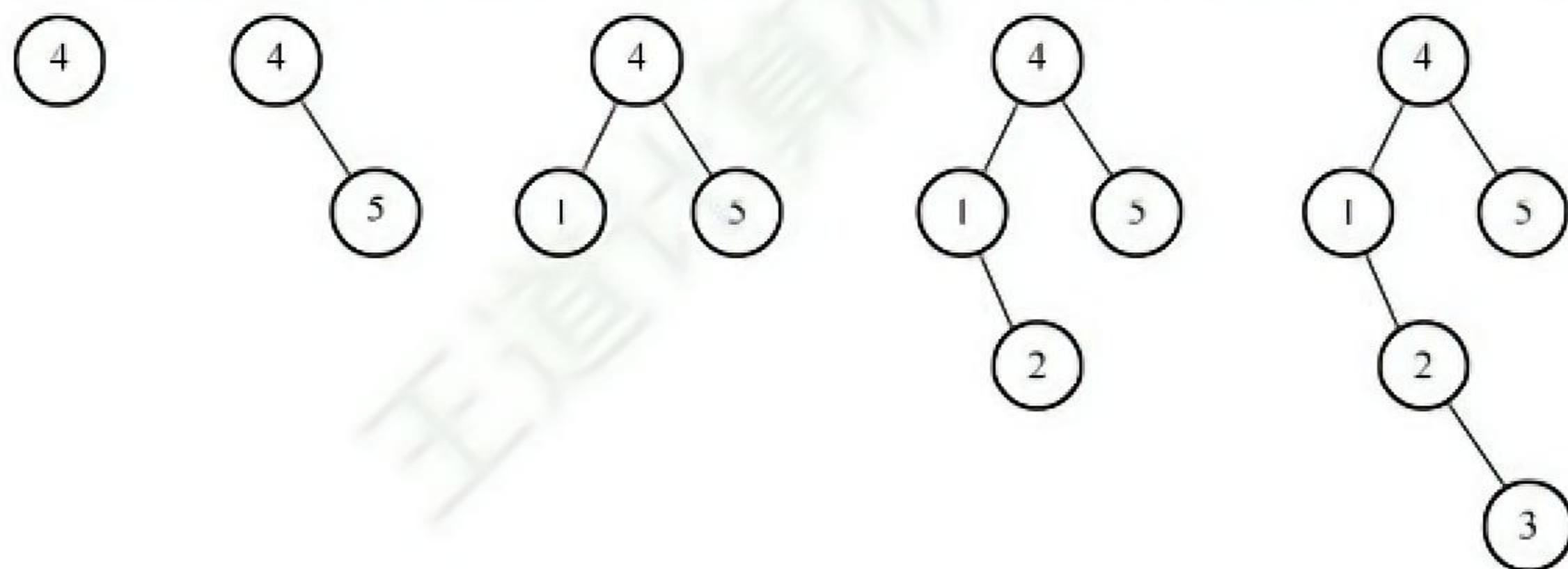


若删除的是 T_1 的非叶结点，且删除和插入操作后导致了平衡二叉树的调整，则该结点有可能通过旋转后继续变成非叶结点， T_1 与 T_3 相同。例如，如下图所示，删除结点 2，用右孩子结点 3 填补，再插入结点 2，平衡二叉树失衡调整，调整后的平衡二叉树和以前相同。



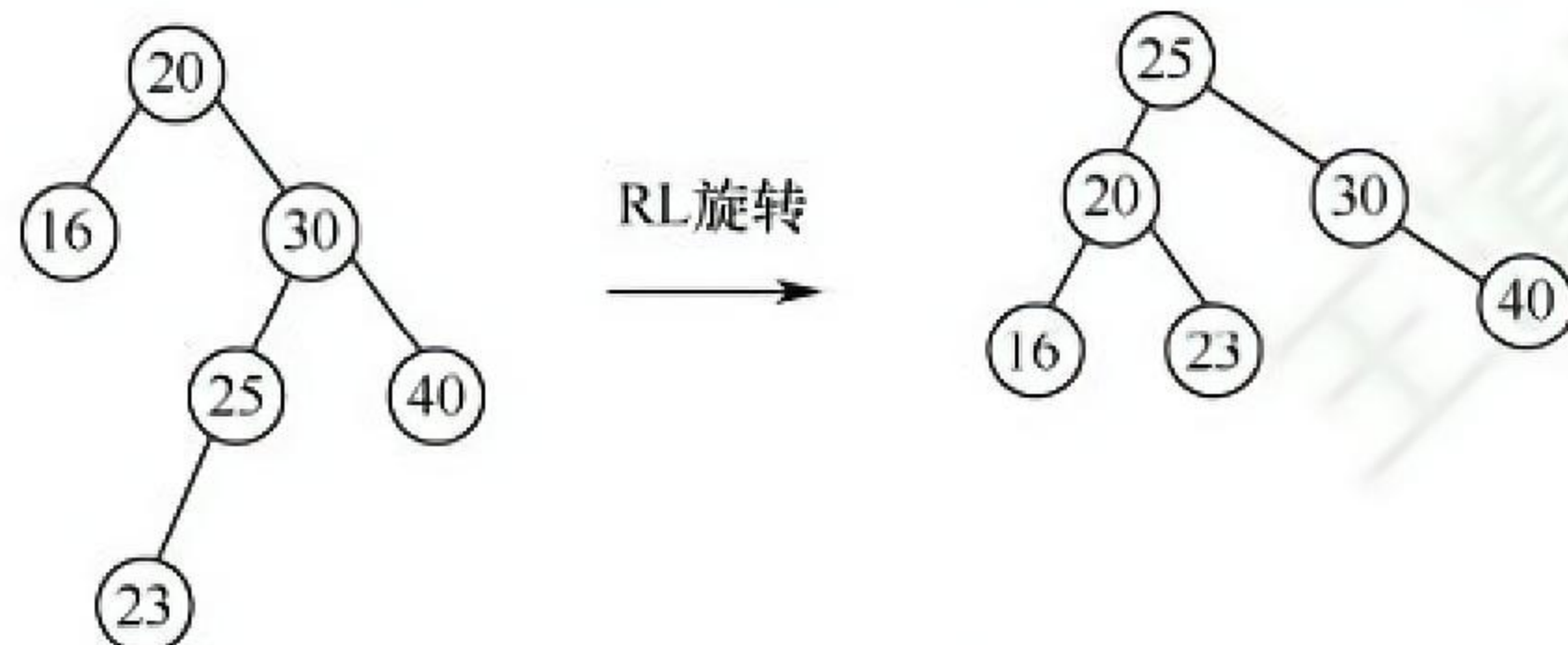
32. B

每个选项都逐一验证，选项 B 生成二叉排序树的过程如下图所示，显然错误。



33. D

关键字 23 的插入位置为 25 的左孩子，此时破坏了平衡的性质，需要对平衡二叉树进行调整。最小不平衡子树就是该树本身，插入位置在根结点的右子树的左子树上，因此需要进行 RL 旋转，RL 旋转过程如下图所示，旋转完成后根结点的关键字为 25，所以选 D。



7.4.4 答案与解析

一、单项选择题

01. D

关键字数目比子树数目少 1，首先可排除 B+树。对于 4 阶 B 树，根结点至少有 2 棵子树（关

键字数至少为 1), 其他非叶结点至少有 $\lceil n/2 \rceil = 2$ 棵子树 (关键字数至少为 1)、至多有 4 棵子树 (关键字数至多为 3)。5 阶 B 树和 6 阶 B 树的分析也类似。题目所示的 B 树, 同时满足 4 阶 B 树、5 阶 B 树和 6 阶 B 树的要求, 因此不能确定是哪种类型的 B 树。

02. C

除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树。对于根结点, 最多有 m 棵子树, 若其不是叶结点, 则至少有 2 棵子树。

03. B

每个非根的内部结点必须至少有 $\lceil m/2 \rceil$ 棵子树, 而根结点至少要有两棵子树, 所以选项 I 不正确。选项 II、III 显然正确。对于 IV, 插入一个元素引起 B 树结点分裂后, 只要从根结点到该元素插入位置的路径上至少有一个结点未滿, B 树就不会长高, 如图 1 所示; 只有当结点的分裂传到根结点, 并使根结点也分裂时, 才会导致树高增 1, 如图 2 所示, 因此选项 IV 错误。

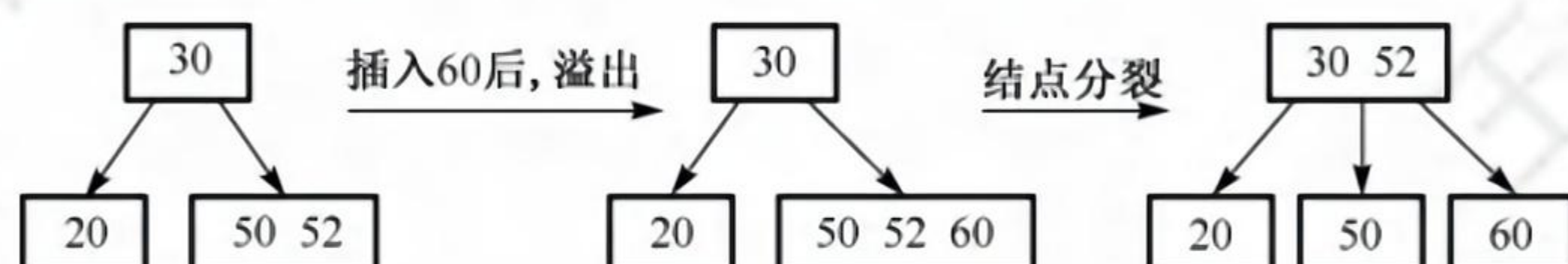


图 1 结点分裂不导致树高增 1 (3 阶 B 树)



图 2 结点分裂导致树高增 1 (3 阶 B 树)

04. B

由于 B 树每个结点内的关键字个数最多为 $m-1$, 所以当关键字个数大于 $m-1$ 时, 则应该分裂。每个结点内的关键字个数至少为 $\lceil m/2 \rceil - 1$ 个, 所以当关键字个数少于 $\lceil m/2 \rceil - 1$ 时, 则可能与其他结点合并 (除非只有根结点)。若将本题题干改为 B+树, 请读者思考上述问题的解答。

05. A

B 树的叶结点对应查找失败的情况, 对有 n 个关键字的查找集合进行查找, 失败可能性有 $n+1$ 种。

06. B、D

由 m 阶 B 树的性质可知, 根结点至少有 2 棵子树; 根结点外的所有非终端结点至少有 $\lceil m/2 \rceil$ 棵子树, 结点数最少时, 3 阶 B 树形状至少类似于一棵满二叉树, 即高度为 5 的 B 树至少有 $2^5 - 1 = 31$ 个结点。由于每个结点最多有 m 棵子树, 所以当结点数最多时, 3 阶 B 树形状类似于满三叉树, 结点数为 $(3^5 - 1)/2 = 121$ (注意, 这里求的是结点数而非关键字数, 若求的是关键字数, 则还应把每个结点中关键字数的上下界确定出来)。

07. D

除根结点外, m 阶 B 树中的每个非叶结点至少有 $\lceil m/2 \rceil - 1$ 个关键字, 根结点至少有一个关键字, 所以总共包含的关键字最少个数 $= (n-1)(\lceil m/2 \rceil - 1) + 1$ 。

注意

由以上题目可知 B 树和 B+树的定义与性质尤为重要, 需要熟练掌握。

08. C、B

5 阶 B 树中共有 53 个关键字，由最大高度公式 $H \leq \log_{\lceil m/2 \rceil}((n+1)/2) + 1$ 得最大高度 $H \leq \log_3[(53+1)/2] + 1 = 4$ ，即最大高度为 4；由最小高度公式 $h \geq \log_m(n+1)$ 得最小高度 $h \geq \log_5 54 \approx 2.5$ ，从而最小高度为 3。

09. A、D

利用前面的公式即最小高度 $h \geq \log_m(n+1)$ 和最大高度 $H \leq \log_{\lceil m/2 \rceil}[(n+1)/2] + 1$ ，易算出最大高度 $H \leq \log_2[(2047+1)/2] + 1 = 11$ ，最小高度 $h \geq \log_3 2048 = 6.9$ ，从而最小高度取 7（注意，有些辅导书针对本题算出的高度要比这里给出的答案多 1，因为它们在 B 树的高度定义中，把最底层不包含任何关键字的叶结点也算进去了）。

10. A

B 树和 B+树的差异主要体现在：①结点关键字和子树的个数；②B+树非叶结点仅起索引作用；③B 树叶结点关键字和其他结点包含的关键字是不重复的；④B+树支持顺序查找和随机查找，而 B 树仅支持随机查找。由于 B+树的所有叶结点中包含了全部的关键字信息，且叶结点本身依关键字从小到大顺序链接，因此可以进行顺序查找，而 B 树不支持顺序查找。B 树和 B+树都可用于文件索引结构，但 B+树更适合做数据库索引和文件索引，因为它的磁盘读/写代价更低。

11. A

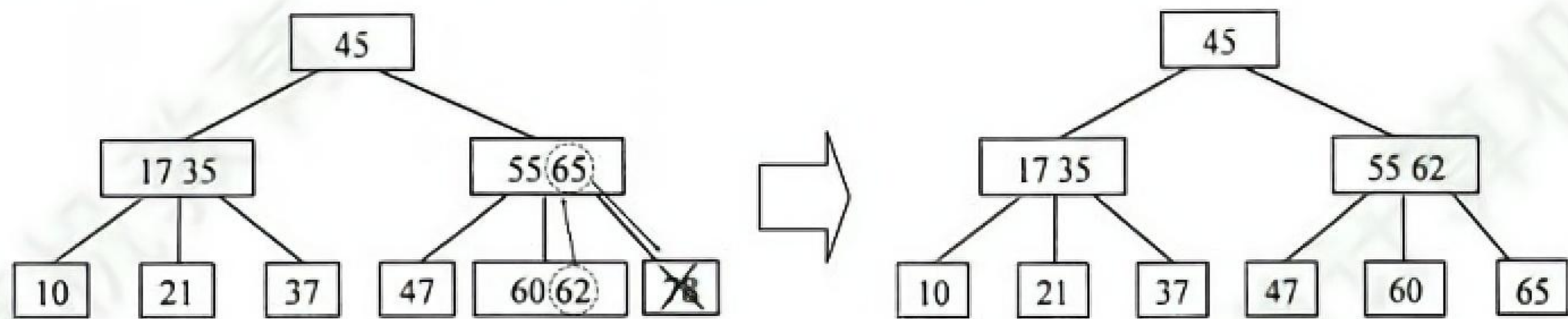
根据 B 树树高 h 的计算公式： $\log_m(n+1) \leq h \leq \log_{\lceil m/2 \rceil}((n+1)/2) + 1$ ，计算得 $h = 5$ （取整数），因为 B 树的根结点已读入内存，所以最多需再启动 4 次磁盘 I/O。

12. D

m 阶 B 树不要求将各叶结点之间用指针链接。选项 D 描述的实际上是 B+树。

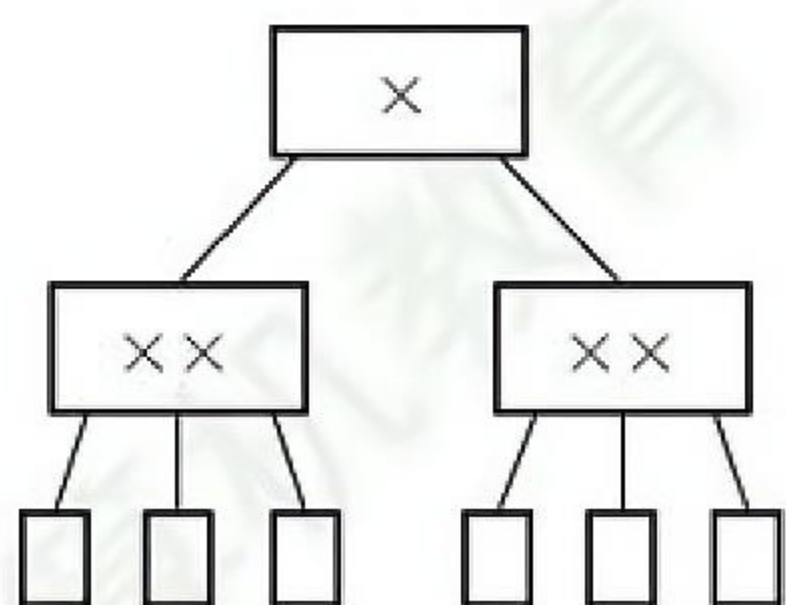
13. D

对于图中所示的 3 阶 B 树，被删关键字 78 所在的结点在删除前的关键字个数 $= 1 = \lceil 3/2 \rceil - 1$ ，且其左兄弟结点的关键字个数 $= 2 \geq \lceil 3/2 \rceil$ ，属于“兄弟够借”的情况，因此要把该结点的左兄弟结点中的最大关键字上移到双亲结点中，同时把双亲结点中大于上移关键字的关键字下移到要删除关键字的结点中，这样就达到了新的平衡，如下图所示。



14. A

对于 5 阶 B 树，根结点的分支数最少为 2（关键字数最少为 1），其他非叶结点的分支数最少为 $\lceil n/2 \rceil = 3$ （关键字数最少为 2），因此关键字个数最少的情况如下图所示（叶结点不计入高度）。



注 意

一般对于某个具体的 B 树图形, 并不能确定是几阶 B 树。对于本题所述的 5 阶 B 树, 不要误认为: “存在至少有一个含关键字结点中的关键字达到 4” 才符合 5 阶 B 树的要求, 因为 5 阶 B 树中各个结点包含的关键字个数最少为 2 ($\lceil 5/2 \rceil - 1 = 2$), 最多为 4 ($5 - 1 = 4$)。当 5 阶 B 树中各个结点包含的关键字个数为 2 时, 也满足 5 阶 B 树的要求。

15. D

关键字数量不变, 要求结点数量最多, 即要求每个结点中含关键字的数量最少。根据 4 阶 B 树的定义, 根结点最少含 1 个关键字, 非根结点中至少含 $\lceil 4/2 \rceil - 1 = 1$ 个关键字, 所以每个结点中关键字数量最少都为 1 个, 即每个结点都有 2 个分支, 类似于排序二叉树, 而 15 个结点正好可以构造一个 4 层的 4 阶 B 树, 使得终端结点全在第四层, 符合 B 树的定义, 因此选 D。

16. A

由于 B+树的所有叶结点中包含了全部的关键字信息, 且叶结点本身依关键字从小到大顺序链接, 因此可以进行顺序查找, 而 B 树不支持顺序查找 (只支持多路查找)。

17. B

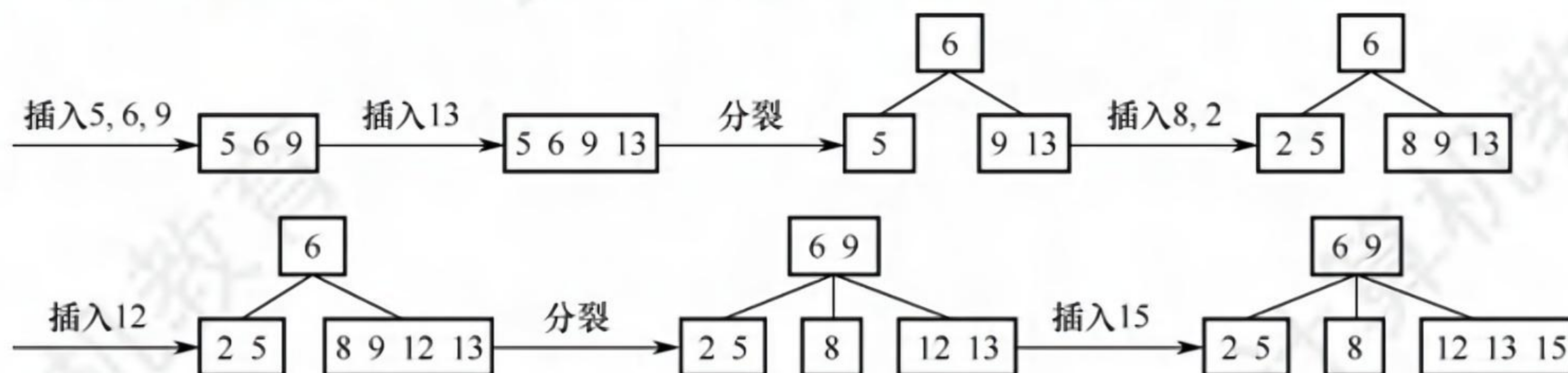
B+树是应文件系统所需而产生的 B 树的变形, 前者比后者更加适用于实际应用中的操作系统的文件索引和数据库索引, 因为前者的磁盘读/写代价更低, 查询效率更加稳定。编译器中的词法分析使用有穷自动机和语法树。网络中的路由表快速查找主要靠高速缓存、路由表压缩技术和快速查找算法。系统一般使用空闲空间链表管理磁盘空闲块。

18. B

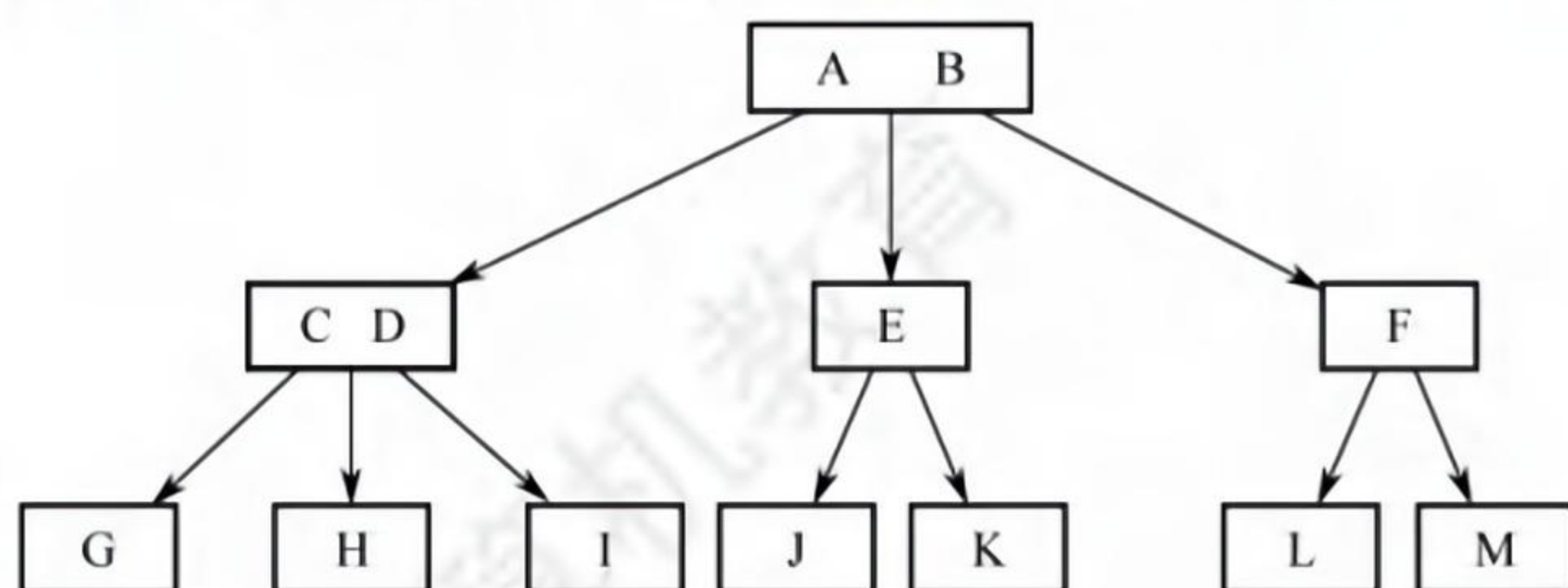
m 阶 B 树的基本性质: 根结点以外的非叶结点最少含有 $\lceil m/2 \rceil - 1$ 个关键字, 代入 $m = 3$ 得到每个非叶结点中至少包含 1 个关键字, 而根结点含有 1 个关键字, 因此所有非叶结点都有两个孩子。此时其树形与 $h = 5$ 的满二叉树相同, 可求得关键字最少为 31 个。

19. B

一个 4 阶 B 树的任意非叶结点至多含有 $m - 1 = 3$ 个关键字, 在关键字依次插入的过程中, 会导致结点的不断分裂, 插入过程如下图所示。得到根结点包含的关键字为 6, 9。

**20. A**

在阶为 3 的 B 树中, 每个结点至多含有 2 个关键字 (至少 1 个), 至多有 3 棵子树。本题规定第二层有 4 个关键字, 欲使 B 树的结点数达到最多, 则这 4 个关键字包含在 3 个结点中, B 树树形如下图所示, 其中 A, B, C, ..., M 表示关键字, 最多有 11 个结点。



21. D

在 5 阶 B 树中，除根结点外的非叶结点的关键字数 k 需要满足 $2 \leq k \leq 4$ 。当被删关键字 x 不在终端结点（最底层非叶结点）时，可以用 x 的前驱（或后继）关键字 y 来替代 x ，然后在相应结点中删除 y 。情况①：删除 260，将其前驱 110 放入 260 处，删除 110 后的结点 $\langle 100 \rangle$ 不满足 5 阶 B 树定义，从左兄弟中借 85，将 85 放入根中，将根中的 90 移入结点 $\langle 100 \rangle$ 变为 $\langle 90, 100 \rangle$ 。情况②：删除 260，将其后继 280 放入 260 处，结点 $\langle 300 \rangle$ 不满足 5 阶 B 树定义且左右兄弟都不够借，结点 $\langle 300 \rangle$ 可以和左兄弟 $\langle 100, 110 \rangle$ 以及关键字 280 合并成一个新的结点 $\langle 100, 110, 280, 300 \rangle$ 。情况③：在情况②中，结点 $\langle 300 \rangle$ 也可以和右兄弟 $\langle 400, 500 \rangle$ 以及关键字 350 合并成一个新的结点 $\langle 300, 350, 400, 500 \rangle$ 。综上， T_1 根结点中的关键字序列可能是 $\langle 60, 85, 110, 350 \rangle$ 或 $\langle 60, 90, 350 \rangle$ 或 $\langle 60, 90, 280 \rangle$ ，仅 D 不可能。

快速解法：假如选项 D 的 60, 90, 110, 350 作为根结点，则在 90 和 110 之间只有 100 这一个数据，显然不符合 5 阶 B 树的定义，因此 D 项不可能。

22. B

B 树的插入操作可能导致叶结点分裂，而叶结点分裂可能导致父结点分裂，若这个分裂过程传导到根结点，则会导致 B 树高度增 1，I 正确。若被删结点是叶结点，则显然会导致叶结点变化；若被删结点不是叶结点，则要先将被删结点和它的前驱或后继交换，最终转换为删除叶结点，还是导致叶结点变化，II 正确。若在非叶结点中查找到了给定的关键字，则不用向下继续查找，III 错误。插入关键字的初始位置是最底层叶结点，但可能因结点分裂而被转移到父结点中，IV 错误。

7.5.6 答案与解析

一、单项选择题

01. B

顺序查找可以是顺序存储或链式存储；折半查找只能是顺序存储且要求关键字有序；树形查找法要求采用树的存储结构，既可以采用顺序存储也可以采用链式存储；散列查找中的链地址法解决冲突时，采用的是顺序存储与链式存储相结合的方式。

02. D

关键字集合与地址集合之间存在对应关系时，通过散列函数表示这种关系。这样，查找以计算散列函数而非比较的方式进行查找。

03. D

冲突（碰撞）是不可避免的，与装填因子无关，因此需要设计处理冲突的方法，I 错误。散列查找的思想是计算出散列地址来进行查找，然后比较关键字以确定是否查找成功，II 错误。散列查找成功的平均查找长度与装填因子有关，与表长无直接关系，III 错误。在开放定址的情形下，不能随便删除散列表中的某个元素，否则可能会导致搜索路径被中断（因此通常的做法是在要删除的地方做删除标记，而不是直接删除），IV 正确。

04. C

在开放定址法中散列到同一个地址而产生的“堆积”问题，是同义词冲突的探查序列和非同义词之间不同的探查序列交织在一起，导致关键字查询需要经过较长的探测距离，降低了散列的效率。因此要选择好的处理冲突的方法避免“堆积”。

05. A

平方探测法采用的增量序列是非线性的，它可以跳过一些已被占用的单元，而不是顺序地探测下一单元，这样能减小冲突的概率，I 正确。散列地址 i 的关键字，和为解决冲突形成的某次探测地址为 i 的关键字，都争夺地址 $i, i+1, \dots$ ，因此不一定相邻，II 错误。III 正确。同义词冲突不等于聚集，链地址法处理冲突时将同义词放在同一个链表中，不会引起聚集现象，IV 错误。

06. A

若有 200 个表项要放入散列表，采用线性探测法解决冲突，限定查找成功的平均查找长度不超过 1.5，则

$$ASL_{\text{成功}} = \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right) \leq 1.5 \Rightarrow \alpha = \frac{200}{m} \leq \frac{1}{2} \Rightarrow m \geq 400$$

07. D

K 个关键字在依次填入的过程中，只有第一个不会发生冲突，所以探测次数为 $1+2+3+\dots+K=K(K+1)/2$ 。

08. C

散列表的平均查找长度与装填因子 α 直接相关，表的查找效率不直接依赖于表中已有表项个

数 n 或表长 m 。若表中存放的记录全是某个地址的同义词，则平均查找长度为 $O(n)$ 而非 $O(1)$ 。

09. D

聚集是因选取不当的处理冲突的方法，而导致不同关键字的元素对同一散列地址进行争夺的现象。用线性探查法时，容易引发聚集现象。

10. C

“下一个空位”可以大于或小于但不等于原散列地址，等于原散列地址是没有意义的。

11. D

由散列函数计算可知，14, 1, 27, 79 散列后的地址都是 1，所以有 4 个记录。

12. A, B

因为在链地址法中，映射到同一地址的关键字都会链到与此地址相对应的链表上，所以探测过程一定是在此链表上进行的，从而这些位置上的关键字均为同义词；但在线性探测法中出现两个同义关键字时，会把该关键字对应地址的下一个地址也占用掉，两个地址分别记为 Addr 、 $\text{Addr}+1$ ，查找一个满足 $H(\text{key})=\text{Addr}+1$ 的关键字 key 时，显然首次探测到的不是 key 的同义词。

13. A, C

H 的取值有 17 种可能，对应到不同的链表中，所以链表的个数应为 17。由于 $H(\text{key})$ 的取值范围是 $0\sim 16$ ，所以数组下标为 $0\sim 16$ 。

14. A

线性探测法的公式为 $H_i = (H(\text{key}) + d_i) \% m$ ，其中 $d_i = 1, 2, \dots, m-1$ 。 $H(49) = 49 \% 11 = 5$ ，有冲突； $H_1 = (H(49) + 1) \% 14 = 6$ ，有冲突； $H_2 = (H(49) + 2) \% 14 = 7$ ，有冲突； $H_3 = (H(49) + 3) \% 14 = 8$ ，无冲突。

15. C

插入过程如下： $H(26)=9$ ，不冲突； $H(25)=8$ ，不冲突； $H(72)=4$ ，不冲突； $H(38)=4$ ，冲突，冲突处理后的地址为 5； $H(8)=8$ ，冲突，冲突处理后的地址为 10； $H(18)=1$ ，不冲突； $H(59)=8$ ，冲突，冲突处理后的地址为 11。因此，在表中查找 59 需要探查 4 次。

16. B

插入过程如下： $6 \% 17 = 6$ ； $22 \% 17 = 5$ ； $7 \% 17 = 7$ ； $26 \% 17 = 9$ ； $9 \% 17 = 9$ ，冲突，平方探测法探测 10（无冲突）； $23 \% 17 = 6$ ，冲突，平方探测法探测 7（冲突），探测 5（冲突），探测 10（冲突），探测 2（无冲突）。因此，关键字 23 应放在位置 2。构造的散列表如下表所示。

地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
元素			23			22	6	7		26	9						

17. C

由于散列函数的选取，仍然有可能产生地址冲突，冲突不能绝对地避免。

18. D

散列表的查找效率取决于散列函数、处理冲突的方法和装填因子。显然，冲突的产生概率与装填因子（即表中记录数与表长之比）的大小成正比，I 与题意相反。II 显然正确。采用合适的冲突处理方法可避免聚集现象，也将提高查找效率，III 正确。例如，用链地址法处理冲突时不存在聚集现象，用线性探测法处理冲突时易引起聚集现象。

19. D

堆积现象因冲突而产生，它对存储效率、散列函数和装填因子均不会有影响，而平均查找长度会因为堆积现象而增大。散列函数是指将关键字映射到哈希地址的函数。存储效率和装填（装

载) 因子的定义相同, 指哈希表中已存储的元素个数与哈希表长度的比值。这些因素都与堆积现象无关, 而只与哈希表的结构和设计有关。

20. C

根据题意, 得到的 HT 如下:

0	1	2	3	4	5	6
	22	43	15			

$$ASL_{成功} = (1 + 2 + 3)/3 = 2。$$

21. C

采用线性探查法计算每个关键字的存放情况如下表所示。

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字	98	22	30	87	11	40	6	20			

由于 $H(key) = 0 \sim 6$, 查找失败时可能对应的地址有 7 个, 对于计算出地址为 0 的关键字 key_0 , 只有比较完 $0 \sim 8$ 号地址后才能确定该关键字不在表中, 比较次数为 9; 对于计算出地址为 1 的关键字 key_1 , 只有比较完 $1 \sim 8$ 号地址后才能确定该关键字不在表中, 比较次数为 8; 以此类推。需要特别注意的是, 散列函数不可能计算出地址 7, 因此有

$$ASL_{失败} = (9 + 8 + 7 + 6 + 5 + 4 + 3)/7 = 6$$

22. D

原题再现。填装因子越大, 说明哈希表中存储的元素越满, 发生冲突的可能性就越高, 导致平均查找长度越大。散列函数、冲突解决策略也会影响发生冲突的可能性。I、II、III 都正确。

23. C

当采用开放定址法时, 不能随便物理删除表中的已有元素, 因为若删除元素, 则可能截断其他具有相同散列地址的元素的查找地址。因此, 当要删除一个元素时, 可给它做一个删除标记。依次将 2022, 12, 25 插入散列表, 然后删除 25, 得到的散列表如下:

地址	0	1	2	3	4
关键字		2022	12		25 (删除)
查找失败次数	1	3	2	1	2

当查找位置是删除标记时, 应继续往后查找。

查找失败的平均查找长度为 $(1 + 3 + 2 + 1 + 2)/5 = 1.8$ 。